



FS

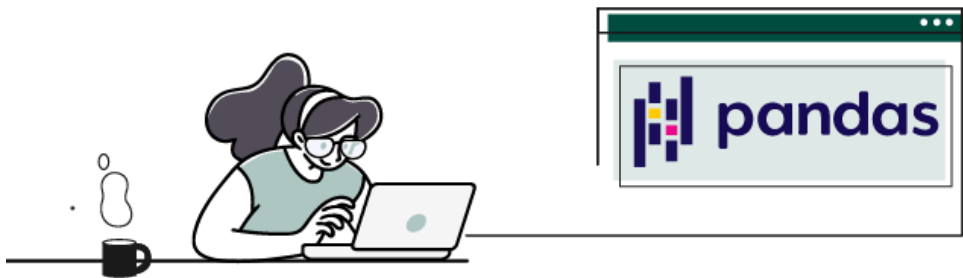


← Découvrez les librairies Python pour la Data Science



19% de progression

Agrégez des données avec Pandas



Maintenant que nous sommes capables d'accéder à tout ce que nous souhaitons au sein de notre data frame, je vous propose de voir comment agréger les différentes informations.

Notre banque opère dans 6 villes différentes : Toulouse, Paris, Marseille, Lyon, Nice et Bordeaux. Chaque prêt contracté par un de nos clients est forcément relié à l'une de ces agences. Et vous risquez de vous poser cette question :

Admettons que je veuille calculer le chiffre d'affaires mensuel réalisé par chaque agence. Comment puis-je faire ?

Bonne question !

Il va donc falloir :

- sélectionner l'ensemble des clients de l'agence de Toulouse et calculer la somme de leurs mensualités ;
- sélectionner l'ensemble des clients de l'agence de Paris et calculer la somme de leurs mensualités ;
- etc.

Le résultat final serait ainsi une ligne par agence/ville, avec la somme

des mensualités perçues par chacune d'elles.

Cette opération est ce qu'on appelle en algèbre relationnelle une **agrégation**. C'est une opération très courante sur des data frames, soit pour analyser les données sous un certain angle, soit pour recalculer



Je vous propose de voir comment réaliser cela avec Pandas.

Agrégez plusieurs lignes

La première méthode pour faire une agrégation avec Pandas est

`.group_by` .

Pour l'utiliser, vous aurez besoin de vous **fixer** sur une ou plusieurs colonnes, qui seront ce qu'on appelle les **index** de votre résultat agrégé.

Cela vous rappelle-t-il quelque chose ?

Eh oui ! Comme nous l'avons dit dans le chapitre précédent, l'index n'est pas forcément l'indice, ça peut être également des chaînes de caractères. Dans notre exemple, notre index serait la variable `ville` . L'index va permettre de créer plusieurs groupes : un pour chaque valeur unique de l'index.

Après avoir choisi les colonnes sur lesquelles vous allez vous fixer, vous aurez à choisir une **fonction d'agrégation**. La fonction d'agrégation va prendre en entrée un groupe de plusieurs lignes, pour effectuer un calcul sur celles-ci dans l'optique de retourner *une unique valeur pour chacun des groupes*. Ici, la fonction d'agrégation serait une somme.

Si vous avez déjà fait un peu de SQL, vous aurez très certainement fait le lien entre cette opération et l'opération de GROUP BY réalisable en SQL. Les deux fonctionnent en effet de façon très similaire !

Considérons le data frame `test` suivant :

```
test
```

	fix	col
0	a	3

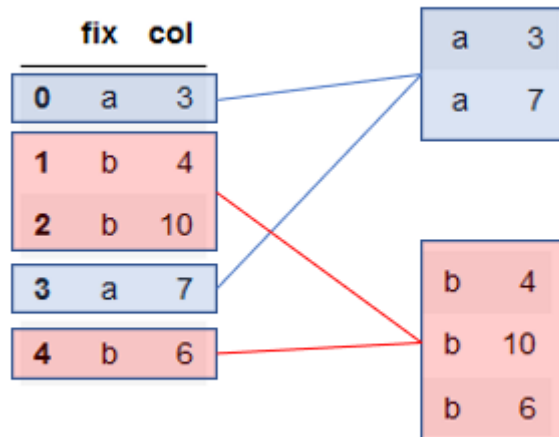


2	b	10
3	a	7
4	b	6

On souhaite calculer la moyenne de la variable `col` pour chaque valeur de la variable `fix`. La ligne en Python pour faire cela est :

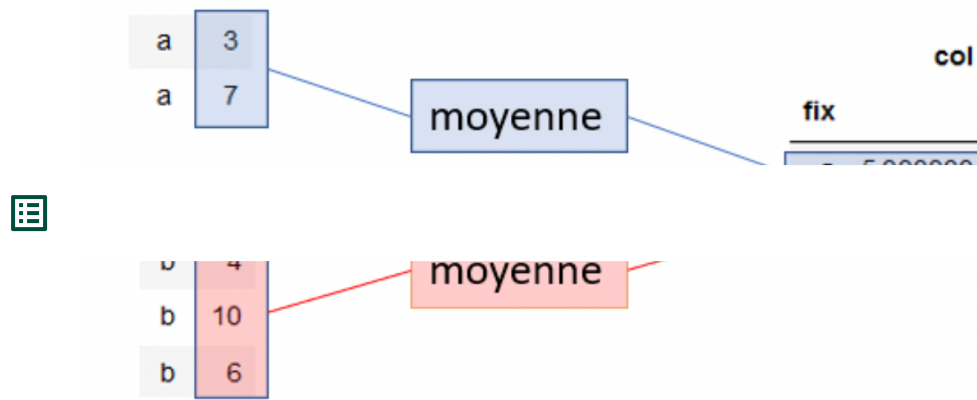
```
1 test.groupby('fix').mean()
```

Décomposons ce qu'il se passe lors de l'exécution de cette ligne. Dans un premier temps, notre index sera la variable `fix` ; il va donc y avoir 2 groupes créés, un pour chaque valeur dans notre variable `fix`.



Application de la fonction `groupby` avec la variable `fix`

Sur chacun de ces groupes, on va appliquer la fonction d'agrégation choisie (ici, la moyenne) afin d'avoir le résultat souhaité :



Application de la fonction mean (moyenne)

À présent, comment calculer l'information recherchée initialement, soit le chiffre d'affaires total de chacune de nos agences ?

```
1 prets.groupby('ville').sum()
```

Maintenant, imaginons que nous ne souhaitons la même information que par agence ET par type de prêt. Vous vous en doutez très probablement, il suffira de transmettre la liste des variables à placer en index :

```
1 prets.groupby(['ville', 'type']).sum()
```

Et si vous ne souhaitez avoir que le résultat sur la variable *remboursement* :

```
1 prets.groupby(['ville', 'type'])['remboursement'].sum()
```

On peut même appliquer des fonctions d'agrégation différentes en fonction de la colonne, voire appliquer plusieurs fonctions d'agrégation sur une même colonne. Ici, je calcule la moyenne et la somme de **remboursement** par agence, ainsi que le maximum de **revenu** :

```
1 prets.groupby('ville').agg({'remboursement': ['sum', 'mean'],
2   'revenu': 'max'})
```

Lorsqu'on utilise **group_by**, les variables fixées en index se retrouvent... en index. C'est-à-dire que si vous essayez d'accéder à la variable **ville** du résultat du *group by*, vous

aurez simplement une erreur ! Vous serez obligé d'effectuer un

`reset_index` pour la replacer en "colonne".

Reprenons la dernière ligne de code présentée. Imaginons à présent



prochaine section.

Agrégez des lignes et des colonnes

Nous souhaitons donc avoir la même agrégation, mais avec cette fois-ci, en lignes nos agences, et en colonnes les différents types de prêts.

La méthode qui vous permettra de faire cela est la méthode

`.pivot_table`. Celle-ci prend 4 arguments en paramètres :

- **index** : variable(s) placée(s) en ligne ;
- **columns** : variable(s) placée en colonne(s) ;
- **values** : variable sur laquelle on va appliquer la fonction d'agrégation ;
- **aggfunc** : fonction d'agrégation.

La méthode permettant la transformation inverse s'appelle `melt`. Je vous propose de découvrir ces deux nouvelles méthodes en vidéo :

03:37

Prêt à mettre tout cela en pratique ? C'est parti !

À vous de jouer



Contexte

Vous l'avez probablement remarqué, certains clients ont contracté plusieurs prêts au sein de notre établissement. Cela fausse donc potentiellement les calculs réalisés jusque-là. Le responsable revient vers vous avec des demandes plus précises.

Consignes

Voici l'e-mail qu'il vous a envoyé :

Hello,

J'aurais plusieurs demandes que je souhaiterais que tu traites dès que tu peux. On en aurait besoin pour le comité de direction prévu en fin de semaine. Pourrais-tu :

- créer un data frame de profil client pour avoir par client toutes les informations qui le concernent, résumées en une ligne ;
- calculer le nombre exact de personnes en état de situation bancaire risquée, à partir du taux d'endettement et de ces profils clients ;
- calculer le bénéfice total dégagé par chacune de nos agences, par type de prêt ;
- calculer les bénéfices moyens réalisés par chaque agence, pour chaque type de prêt, sous forme de tableau à double entrée ;
- me communiquer la ville qui semble la plus intéressante pour y

développer les prêts immobiliers ?

Merci d'avance.

Je vous invite réaliser cet [exercice](#).



Félicitations pour être arrivé au bout ! Voici [la correction correspondante](#).

En résumé

- Pour agréger des données, il faut définir une ou plusieurs variables placées en index, pour former des groupes sur lesquels s'appliquera une fonction d'agrégation.
- Le résultat final d'une agrégation contiendra autant de lignes que de valeurs différentes dans la variable ou les variables choisies en index.
- Il existe deux méthodes en Python permettant de réaliser une agrégation :
 - la méthode `group_by` ;
 - la méthode `pivot_table` .

Je vous propose maintenant de voir comment fusionner plusieurs fichiers de données, avec Pandas.



[Avez-vous une suggestion pour nous ?](#)

Chapitre suivant →